

CodeDiff

CI · [Latest release](#) · [docs.rs](#) · [coverage](#) · License: AGPL v3+

Fast, robust, syntax-aware code diffing.

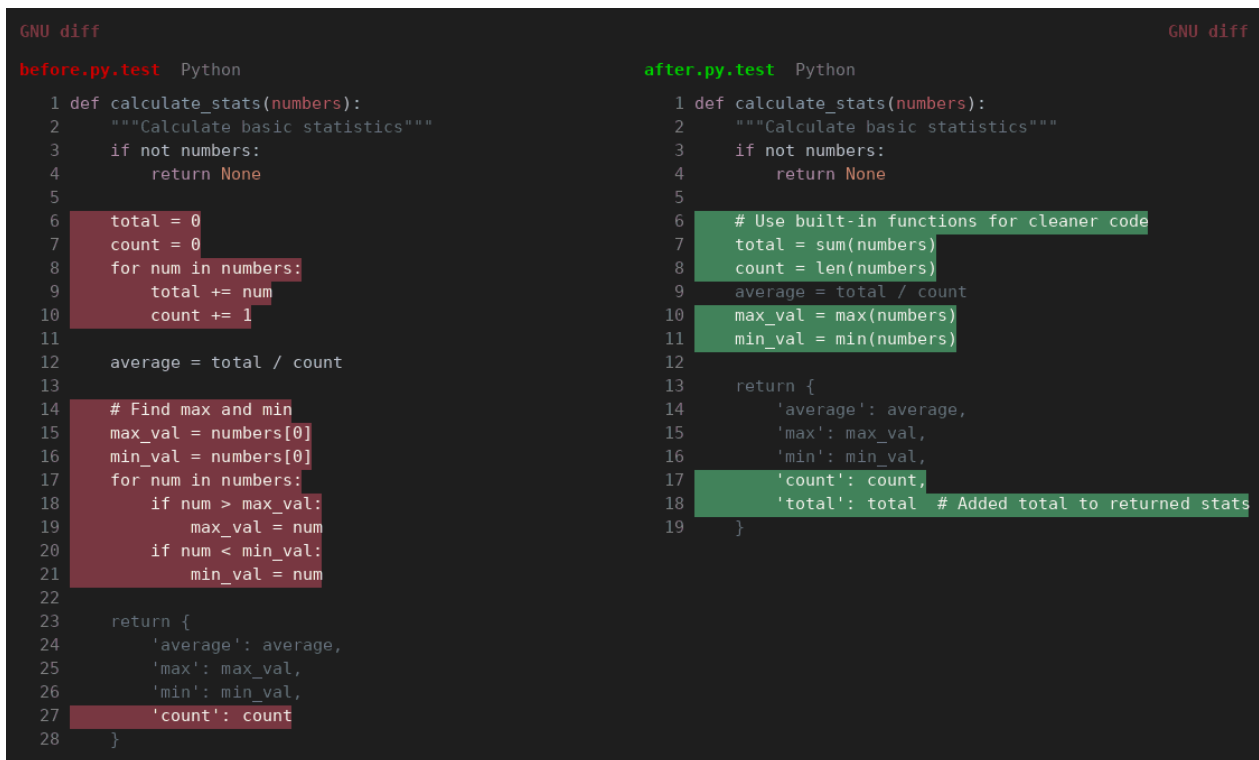


Figure 1: An animation of one Python refactoring painted two ways. A vertical bar sweeps left to right and back across a two-pane diff. On one side of the bar, GNU diff marks whole lines as deleted and inserted; on the other, codediff paints only the parts that changed - `sum(numbers)` and `len(numbers)` rather than the whole assignment, and numbers shown as moved rather than rewritten.

Two hand-rolled loops become `sum`, `len`, `max` and `min`, and the bar sweeps between the two readings of that one change. `diff` can only mark whole lines, so it calls the assignments deleted and reinserted; `codediff` leaves `total =` alone and paints what the right-hand side became, matching numbers in `max(numbers)` to the `numbers[0]` it came from.

Try it in the browser: twenty real changes in `codediff`'s own viewer, each shown as Unix `diff` marks it and as `codediff` maps it, one click apart.

The same change in the terminal UI, which is what `codediff` opens with when you give it no arguments:

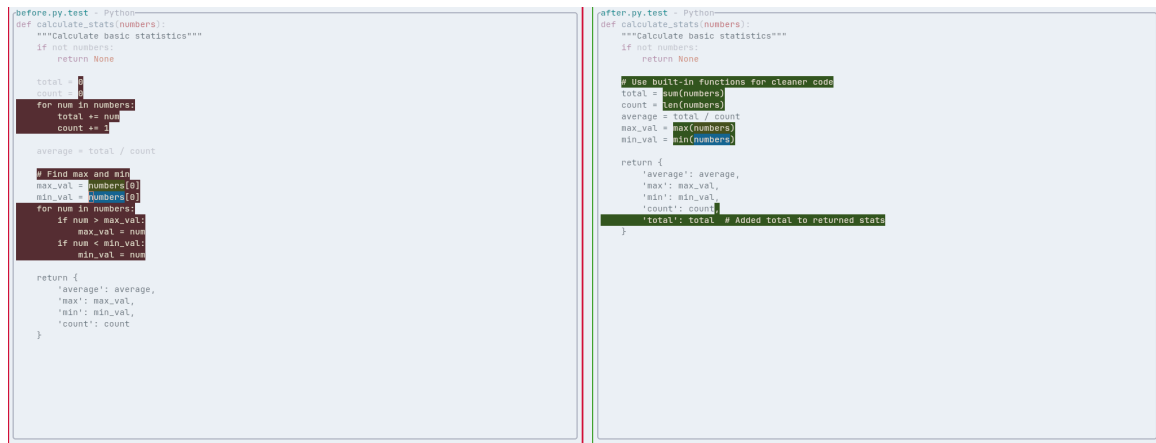


Figure 2: A screenshot of codediff's two-panel terminal UI showing the same Python refactoring, with the changed right-hand sides highlighted rather than whole lines

Installation

```
cargo install codediff
```

This command builds CodeDiff from source. You need a C compiler on PATH and a Rust toolchain, edition 2024 or later (rustc 1.88 or later). The build compiles every tree-sitter grammar from C. The first `cargo install` takes a few minutes, because of this and the `lto = "fat"` release profile.

Pre-built binaries for Linux, macOS (Intel and Apple Silicon), and Windows are attached to every [GitHub release](#), along with unofficial `.deb` packages for Debian and Ubuntu, a `codediff-completions-and-man.tar.gz` holding the man page and shell completions, and a `SHA256SUMS.txt` covering all of them.

Debian and Ubuntu

Those `.deb` packages are also served as a signed apt repository, so `apt upgrade` picks up new versions like any other package. amd64 and arm64:

```
sudo install -d -m 0755 /etc/apt/keyrings
curl -fsSL https://ivankovic.github.io/codediff/apt/codediff-archive-keyring.gpg \
| sudo tee /etc/apt/keyrings/codediff-archive-keyring.gpg > /dev/null
echo "deb [signed-by=/etc/apt/keyrings/codediff-archive-keyring.gpg] \
https://ivankovic.github.io/codediff/apt stable main" \
| sudo tee /etc/apt/sources.list.d/codediff.list > /dev/null
sudo apt update && sudo apt install codediff
```

The repository is **unofficial** - it is built and signed by this project, not by Debian or Ubuntu, and nothing in it has been through either distribution's review. `signed-by` is what keeps that scoped: the key above can vouch for codediff and for nothing else on your system. Read [packaging/README.md](#) for how the packages are built and why a package in the Debian archive proper is not a reachable goal.

It carries the last five releases, so `apt install codediff=<version>` can still reach a previous one. Older versions stay attached to their own GitHub release.

On NixOS, or anywhere with Nix installed, no installation step is needed at all:

```
nix run github:ivankovic/codediff
```

Recipes for Arch (AUR), Gentoo, Debian and Nix live in [packaging/](#). None of them are submitted to their respective repositories yet - unlike the VS Code extension, which is published; see [Editor integration](#).

Shell completions and the man page

```
codediff util completions bash # also: zsh, fish, powershell, elvish
codediff util man
```

Both print to stdout, generated from the same definition as `--help`, so they cannot drift from the real flag list. Every package under `packaging/` installs them for you; if you installed a pre-built binary by hand, redirect them wherever your shell expects — or `source <(codediff util completions bash)` for the current session.

The git-history analysis tools in `src/bin/` sit behind an off-by-default `stats` feature. `cargo install` does not install these tools. They matter only if you build from a checkout. The `stats` feature exists because these tools need `git2` and its own `OpenSSL/libssh2` build dependencies. The diffing tool itself does not need these dependencies. Build these tools with `cargo build --features stats`.

Editor integration

- **VS Code** - [codediff-vscode](#), v0.0.1. Search for **CodeDiff** in the Extensions view, or code `--install-extension ivankovic.codediff`. Also on [Open VSX](#) for VSCodium, Cursor and Windsurf. It paints codediff's verdict onto real editors as decorations, and adds five commands: diff two files, diff against HEAD, against any revision, against the last save, and clear the highlights. The binary ships with the extension, per platform.
- **Neovim** - [codediff.nvim](#).

Both drive `codediff --mode json BEFORE AFTER`, which prints one JSON object describing each side's changed ranges, their operation (insert/delete/update/move), a move's real counterpart range in the other file, and the nearest enclosing declaration - enough to place highlights on buffers the editor already has open, with no ANSI parsing. The schema is documented at the top of `src/tui/json_output.rs`. **Its columns are byte offsets**, which is what tree-sitter reports and what Neovim consumes directly; an editor using UTF-16 (VS Code) or character offsets must convert per line, which is what the VS Code extension does and tests.

Using CodeDiff

The interactive TUI

`codediff` with no arguments opens a terminal UI with two panels, "Before" and "After". Terminals 220 columns or wider show both panels side by side. Narrower terminals show one panel at a time. `codediff BEFORE AFTER`, with two file paths, opens directly into their diff, not an empty viewer. If you build from a checkout instead of `cargo install`, use `cargo run --` in place of `codediff` in every example below.

- `Tab` — switch the active panel. `v` cycles the layout between auto, forced dual, and forced single, and `codediff` remembers the choice across runs.

- `o` — open a file selector for the active panel. Type to filter the listing; Backspace widens the filter, and only with nothing left to widen does it go to the parent directory. `Ctrl-h` toggles dotfiles. Once both panels have a file, `codediff` computes and draws the diff between them automatically, color-coded by change type (inserted, deleted, updated, moved) using the current overlay theme - see `c` below. On the empty start screen, the digit keys 1-9 reopen a recently diffed pair.
- `r` — reload both files from disk and re-diff, keeping the cursor where it is. `e` opens the focused panel's file in `$VISUAL/$EDITOR` at the cursor line and re-diffs on return - together they close the read-diff/fix-code loop without leaving the session. While a diff is computing, `Esc` cancels it and keeps the previous result.
- `c` — open the color theme picker. Built-in themes: Dark (default), Solarized Dark, Solarized Light, Dracula, Nord, Gruvbox Dark, Monokai, One Dark. Moving the selection previews the theme live; `Enter` keeps it, and `codediff` remembers your choice across runs.
- `?` — show every keybinding plus a color legend rendered in the active theme's actual colors, and copyright/license/repository info. `j/k` scrolls it. `?` or `Esc` closes it.
- Arrow keys or `h/j/k/l` — move the cursor, one line or column at a time, same as a text editor. The range under the cursor, and the matching range on the other panel, highlight whenever it's part of a real change; unchanged (identical) content is never highlighted. The other panel's cursor always follows the matched node. `Enter` jumps to the matched counterpart on the other panel (and back). Both panels carry a line-number gutter, long lines scroll horizontally with the cursor (... marks a cut edge), and a one-column strip at each panel's right edge maps where the changes are in the whole file.
- `n/p` — jump straight to the next or previous change. This skips unchanged lines entirely. It wraps around at the start or end of the file. `g` jumps to a line number.
- `/` — search the focused panel for text (smart-case: all-lowercase matches insensitively, any capital matches exactly). The match count updates live while typing, and matches highlight in the theme's search color. `Enter` jumps to the nearest match; a bare `Enter` repeats the last search. `Esc` cancels; an empty query clears the current search's highlights. `>/<` step to the next/previous match once a search is active.
- `Ctrl-d/Ctrl-u` — move the cursor half a page. `Ctrl-e/Ctrl-y` — scroll the view one line without moving the cursor. `Page Up/Page Down/Home/End` — scroll.
- `s` — toggle syntax highlighting.
- Mouse — the wheel scrolls the panel under the pointer; a left click focuses that panel and places the cursor on the clicked character. Terminal-native text selection stays available via your terminal's usual modifier (typically Shift-drag).
- `q` or `Esc` — quit. If a dialog is open, `Esc` cancels the dialog instead; while a diff is computing, `Esc` cancels the computation.

The web UI

`codediff-web` is the same viewer served to a browser tab. It is a second binary behind the off-by-default web feature:

```
cargo install codediff --features web
codediff-web BEFORE AFTER
```

It starts a local server, prints the URL, and opens your browser (`--no-open` to skip that, `--port` and `--host` to pick where it listens - the default is a random port on 127.0.0.1). With no arguments it starts empty, like the TUI, and it accepts `git`'s `GIT_EXTERNAL_DIFF` argument list too. Every key from the list above works in the page, `?` included, and the two front ends share one config file: a theme, layout or render option chosen in one is what the other starts with.

Three things differ because a browser is not a terminal. `e` opens `$VISUAL/$EDITOR` in the terminal `codediff-web` was started from (a GUI editor opens wherever it opens), and the page re-diffs when it exits. `q` stops the server, the way it quits the TUI; closing the tab does not, so a reload does not end the session. `Ctrl-Z` has no counterpart. Only the machine running `codediff-web` can use the page: it binds a loopback address, and every request has to carry a token the page got when it loaded, so no other site open in the same browser can read a diff or drive the session. Listening on anything but a loopback address exposes your files to whoever can reach it.

Headless / batch mode

`codediff --headless BEFORE AFTER`, or its synonym `--batch`, prints the diff as plain text, with optional color, instead of opening the TUI. Use this for scripts, CI, or any case where stdout is not a real terminal. Headless mode also starts automatically whenever stdout is not a terminal, for example when piped into `less` or redirected to a file. Because of this, `codediff BEFORE AFTER | less` works without the flag.

Every printed line is prefixed with its line number, so the moved-chunk headers' "Moved to lines 40-60" cross-references can actually be followed. `codediff` collapses long runs of unchanged lines. It keeps 3 lines of context on each side of a change (override with `--context N`), the same convention as `diff -u`. `codediff` also prefixes each hunk with the nearest enclosing function, class, or struct line, when that line is not otherwise visible. This shows the location of a change deep inside a large file. The reader does not need to see the whole file around it.

Colors are on by default (git's pager renders them); pass `--color never`, or set `NO_COLOR=1` (see <https://no-color.org>), to disable ANSI colors, for example when you redirect output to a file - `--color always` forces them even under `NO_COLOR`.

`codediff` exits 0 on success and 2 on error. For scripting, pass `--exit-code` to additionally get 1 when the files differ, the `diff(1)` convention. That is opt-in rather than the default for the same reason `git diff` exits 0 even when files differ: `codediff`'s usual non-interactive callers are version control systems driving it as a display tool, and they read a non-zero exit as "the tool failed" - `jj` warns on every file, and `git difftool` with `difftool.trustExitCode=true` aborts the whole diff. (The 7-argument `GIT_EXTERNAL_DIFF` form stays at 0 even with `--exit-code`, since git treats a non-zero exit there as fatal.)

Git integration

`codediff` doubles as a `git difftool` backend. Run the interactive setup wizard, which asks whether to configure it globally or for the current repository only:

```
codediff git configure
```

Or configure it by hand:

```
git config difftool.codediff.cmd 'codediff "$LOCAL" "$REMOTE" '  
git difftool --tool=codediff
```

Run `git config diff.tool codediff` to make plain `git difftool` use `codediff` by default, without needing `--tool`. If you do not want git to ask "view diff ... [Y/n]?" before every file, run `git config difftool.prompt false`.

git difftool opens the interactive TUI. git diff and git log -p never do. `git diff` pipes its output through git's pager, and a full-screen TUI cannot draw onto a pipe, so `codediff`

always falls back to plain text there regardless of terminal or `GIT_EXTERNAL_DIFF` config (see "Headless / batch mode" above). If you want the interactive viewer from git, use `git difftool`, not `git diff`. If `git difftool` still doesn't open interactively over SSH, reconnect with `ssh -t` — the session needs an allocated pseudo-terminal; tmux panes always have one, so tmux itself is never the cause.

codediff also works directly with `git diff` and `git log -p`, through `GIT_EXTERNAL_DIFF`. This path needs no `difftool` config:

```
GIT_EXTERNAL_DIFF=codediff git diff
```

Files with no tree-sitter grammar (an unrecognized extension, or none at all - Makefile, Dockerfile, ...) fall back to a plain line-level diff instead of the syntax-aware one, so a change touching one of them never blocks `git diff` from showing the rest.

Binary files - anything codediff cannot read as text, a PDF or an image - get a one-line Binary file <path> differs notice instead of a diff, the same stand-in git and `diff(1)` print for them. They likewise never block the rest of a `git diff`: an external diff that exits non-zero makes git abandon the *entire* run, so codediff reports an unshowable file as a successful diff of nothing rather than as a failure.

Jujutsu (jj) integration

jj does not read git's `difftool/diff.external` settings, even in a colocated repo, so it needs its own configuration. Run the setup wizard:

```
codediff jj configure
```

Or configure it by hand:

```
jj config set --user merge-tools.codediff.program codediff
jj config set --user merge-tools.codediff.diff-args '["$left","$right"]'
jj config set --user merge-tools.codediff.diff-invocation-mode file-by-file
```

That registers `jj diff --tool codediff`. To make it the default for plain `jj diff` as well:

```
jj config set --user ui.diff-formatter codediff
```

Use `--repo` in place of `--user` to configure the current repository only.

diff-invocation-mode = "file-by-file" is required. jj's default hands a diff tool two *directory* trees; codediff diffs two files, so without this setting every invocation fails. With it, jj passes one changed file pair at a time, keeping each file's real path and extension, so language detection works exactly as it does under git.

`jj diff` runs its formatter under a pager, so codediff renders in its non-interactive text mode there - the same output `git diff` gets. jj has no equivalent of `git difftool`'s interactive per-file viewer (its terminal-attached hook, `ui.diff-editor`, is for `jj diffedit`/`jj split`, which edit the right-hand side and read it back - not something a read-only viewer should claim to do), so for the full-screen TUI on a jj repo, run `codediff BEFORE AFTER` directly.

Guiding principles

Robust

CodeDiff must process 100% of all commits in the full test dataset.

The full test dataset holds the git commit history of about 7,400 open-source git repositories, as available on the main branch. This list of repositories comes from the Gentoo Linux distribution. Find it in `list_of_repositories.csv`.

Measured once in full, on 2026-09-19: every modified code file in the most recent 50 commits of each of those repositories, 442,530 readable before/after pairs in 25 languages, diffed with no size cap under a 120-second budget and a 6 GB memory cap per process.

442,322 completed (99.95%), with no panics. The largest completed pair holds 7.1 million AST nodes on each side; the corpus's largest file has 23.6 million, and the files of that order the run met - all generated tables - needed more memory than the cap. So the goal is not met at the very top, and the paper says so. 96 pairs ran past the budget and 112 past the memory cap; the latter are twenty generated or embedded files - tree-sitter parser tables, codegen, minified bundles, a PNG as a C array - plus one commit of a 40,000-line single-header C++ library. Given 24 GB, eleven of those files complete. The run, its harness and every pair that did not complete are documented in `research/data/performance/PROVENANCE.md`.

A smaller list of 100 repositories, the "small" dataset, is available in the same directory. Use it for faster iteration when you debug.

Fast

- **A median diff in 100ms or less.**
- **A 99th-percentile diff in 1000ms or less.**

Both are met. Over the 2,001 fixtures in `src/test/data/diffs/`, measured 2026-09-18 on a release build: **p50 7.6ms, p90 78.7ms, p99 347ms**, slowest 1,355ms. **100ms is the 92.7th percentile** — 146 of 2,001 fixtures take longer than that, and 2 take longer than a second.

Two things that number is not. It is not the full dataset the *Robust* goal above names: a diff of every commit in 7,400 repositories is not something this project runs per change, and the fixture corpus is the proxy it uses instead. And the proxy is a pessimistic one, because it is grown deliberately toward hard cases. The one full-corpus run above says how pessimistic: over 442,322 real file modifications the median is 12.2ms and the 99th percentile 2.1s, so the first goal holds on real commits with room to spare and the second does not, on the roughly 1% of files that are generated tables and bundles rather than code anyone edits by hand.

In code, I accept less readable, more complex code, if that code is faster.

Benchmarks make sure that performance does not regress. `make benchmark-quality` prints the distribution above; `make check-quality` compares it against the committed baseline on every push.

Accurate

CodeDiff must match a human's own reading of a change, measured against the hand-authored ground-truth mappings in `src/test/data/diffs/`:

- **90% of test cases with zero mismatched visible nodes.**

- **99% of test cases with at most 1% of visible nodes mismatched.**

Visible is the load-bearing word. Most AST nodes are structure the reader never sees on their own - a block, an `argument_list`, a `declaration_list`, whose every readable byte belongs to some descendant. Getting one of those wrong does not put anything wrong in front of the reader, so counting it alongside a wrongly-matched identifier overstates how wrong the diff is.

A node is visible when it carries text of its own: a leaf, or an interior node with non-whitespace content its children don't cover (a comment whose `//` marker is a separate child, say). This is a property of the syntax tree and the source bytes **and of nothing else** - `codediff::diff::nodes::is_structurally_visible` is the definition. In particular it does not depend on the diff: a measurement whose own denominator moves when the algorithm changes cannot be used to judge the algorithm. Corpus-wide about 68% of nodes are visible.

make benchmark-quality reports both the raw and the visible mismatch count per fixture.

License

Copyright (C) 2026 Marko Ivankovic

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation.

See the LICENSE file for the full text of the License.

Cannot use AGPL software?

Contact me for options.

AI policy

This project uses substantial AI assistance, currently Claude Code. Most commits disclose this with a Co-Authored-By trailer and a link to the session that produced them. This project does not hide that fact. This project does not treat AI assistance as a lesser way to write software.

AI-assisted contributions are welcome. Use whatever tools help you do good work. Disclose your use of these tools the same way. You are still responsible for understanding and standing behind whatever you submit.

For Developers, human or otherwise

See [CONTRIBUTING.md](#) for the technology overview, code-quality and testing expectations, project structure, and what CI checks on every push and PR. [AGENTS.md](#) has additional AI-agent-specific conventions.